

Actividad integradora

Pokédex con PokéAPI

Práctica aplicable a Desarrollo Web y Aplicaciones Móviles

Propósito: desarrollar una aplicación funcional que consuma una API pública, organice información en una interfaz clara y demuestre habilidades de componentes, navegación, estado, persistencia local, manejo de errores y buenas prácticas de código.

Elemento	Descripción
Duración	4 clases de 2 horas cada una
Modalidad	Individual
API base	https://pokeapi.co/api/v2
Producto final	Aplicación web o móvil tipo Pokédex
Evaluación	Entregables diarios + demo final + revisión de código

1. Contexto de la práctica

PokéAPI es una API pública de tipo REST que expone información en formato JSON sobre Pokémon, tipos, habilidades, movimientos, estadísticas e imágenes. En esta práctica se utilizará como fuente de datos externa para construir una aplicación completa, pero acotada al tiempo disponible.

2. Objetivo general

Construir una Pokédex funcional que permita listar Pokémon, consultar detalles, buscar, filtrar, guardar favoritos y comparar estadísticas, aplicando una estructura de proyecto clara y buenas prácticas de desarrollo.

3. Tecnologías permitidas

Materia	Stack base	Herramientas sugeridas
Desarrollo Web	React + Vite + TypeScript	React Router, Axios o Fetch, localStorage, CSS/Tailwind/shadcn opcional
Aplicaciones Móviles	Expo + React Native + TypeScript	React Navigation, Axios o Fetch, AsyncStorage, FlatList
Bonus	NestJS	Proxy/cache hacia PokéAPI, manejo de CORS, endpoints propios

4. Alcance funcional obligatorio

ID	Requerimiento	Descripción
RF01	Listado de Pokémon	Mostrar tarjetas con nombre, imagen, número y tipos.
RF02	Detalle de Pokémon	Consultar y mostrar información completa de un Pokémon seleccionado.
RF03	Búsqueda	Buscar Pokémon por nombre.
RF04	Filtros	Filtrar Pokémon por tipo.
RF05	Favoritos	Agregar y quitar Pokémon favoritos con persistencia local.
RF06	Comparador	Comparar las estadísticas base de dos Pokémon.
RF07	Estados de interfaz	Mostrar carga, error y estado sin resultados.

5. Requerimientos técnicos obligatorios

ID	Requerimiento
RT01	Usar TypeScript.
RT02	Separar la lógica de consumo de API en servicios.
RT03	Crear componentes reutilizables.
RT04	Definir tipos o interfaces para las respuestas principales.
RT05	Manejar errores de red o respuestas no válidas.
RT06	Usar Git con commits claros y progresivos.
RT07	La interfaz debe ser responsive en web o adaptada correctamente a móvil.
RT08	No colocar toda la lógica en un solo archivo o componente.

6. Estructura sugerida del proyecto

Estructura base sugerida:

```
src/  
  components/  
  screens/ o pages/  
  services/  
  hooks/  
  types/  
  utils/  
  storage/
```

7. Endpoints sugeridos

Uso	Endpoint relativo	Propósito
Listado	GET /pokemon?limit=30	Obtener una lista inicial de Pokémon.
Detalle	GET /pokemon/{id or name}	Obtener información completa de un Pokémon.
Tipos	GET /type	Obtener catálogo de tipos.
Pokémon por tipo	GET /type/{id or name}	Obtener Pokémon asociados a un tipo.

8. Entregables por clase

Clase 1: Planeación, estructura y primer consumo de API

- Crear el repositorio y proyecto base.
- Crear estructura de carpetas.
- Crear servicio inicial para consumir PokéAPI.
- Mostrar al menos 20 Pokémon en pantalla.

Tipo	Entrega del día 1
Código	Proyecto inicial ejecutándose sin errores.
UI	Pantalla principal con listado básico de Pokémon.
Evidencia	Commit inicial, proyecto comprimido y captura de pantalla del avance.

Clase 2: Componentes, detalle y navegación

- Crear componente reutilizable para tarjeta de Pokémon.
- Implementar navegación hacia detalle.
- Consultar endpoint de detalle por id o nombre.
- Mostrar imagen, tipos, peso, altura, habilidades y estadísticas base.

Tipo	Entrega del día 2
Código	Componentes separados y servicio de detalle.
UI	Listado y pantalla de detalle funcional.
Navegación	Rutas o pantallas correctamente conectadas.
Evidencia	Commit del día, proyecto comprimido y captura del detalle de un Pokémon.

Clase 3: Búsqueda, filtros y favoritos

- Agregar búsqueda por nombre.
- Agregar filtro por tipo.
- Crear sistema de favoritos.
- Persistir favoritos localmente.
- Agregar estados de carga, error y sin resultados.

Tipo	Entrega del día 3
Código	Búsqueda, filtro y favoritos implementados.
Persistencia	Favoritos guardados en localStorage o AsyncStorage.
UX	Mensajes de carga, error y sin resultados.
Evidencia	Commit del día, proyecto comprimido y captura de favoritos/filtros.

Clase 4: Comparador, cierre visual y demo final

- Implementar comparador de dos Pokémon.
- Mostrar comparación de estadísticas base.
- Pulir diseño, espaciado, colores y responsividad.
- Preparar demo técnica de máximo 5 minutos.
- Subir entrega final al repositorio.

Tipo	Entrega del día 4
Código	Aplicación completa y ejecutándose sin errores.
Funcionalidad	Comparador funcionando con dos Pokémon.
Presentación	Demo de 5 minutos por equipo.
Entrega final	Repositorio actualizado, proyecto comprimido, README y evidencias.

9. Requisitos específicos para la entrega final

- Repositorio Git con historial de commits.
- README con instrucciones para instalar y ejecutar el proyecto.
- Capturas de pantalla de las principales vistas.
- Aplicación funcional sin errores críticos en consola.
- Código organizado por componentes, servicios, tipos y pantallas.
- Demo final donde se muestre el flujo completo de la aplicación.

10. Criterios de evaluación

Criterio	Puntos
Consumo correcto de PokéAPI	20
Listado y detalle funcional	20
Búsqueda, filtros y favoritos	20
Comparador de Pokémon	15
Diseño, UX y responsividad/adaptación móvil	10
Código limpio, estructura y TypeScript	10
Presentación final	5
Total	100

11. Puntos extra opcionales

Bonus	Puntos extra
Backend NestJS como proxy hacia PokéAPI	+10
Cache de respuestas en backend o frontend	+5
Modo oscuro	+5
Gráfica visual de estadísticas	+5
Paginación o scroll infinito	+5
Pruebas unitarias básicas	+5
Diseño por colores según tipo de Pokémon	+5

12. Reglas de trabajo

1. Cada alumno debe trabajar con control de versiones desde la primera clase.
2. No se permite entregar código copiado sin entenderlo o sin poder explicarlo.
3. Cada alumno debe explicar al menos una parte técnica durante la demo.
4. El proyecto debe ejecutarse siguiendo únicamente las instrucciones del README.
5. La entrega final debe estar terminada antes de la presentación en la cuarta clase.

13. Checklist final para alumnos

Estado	Elemento a verificar
☐	El proyecto instala dependencias correctamente.
☐	El listado de Pokémon carga correctamente.
☐	El detalle de Pokémon funciona.
☐	La búsqueda por nombre funciona.
☐	El filtro por tipo funciona.
☐	Los favoritos se guardan y se mantienen al recargar o cerrar la app.
☐	El comparador funciona con dos Pokémon.
☐	La aplicación muestra estados de carga y error.
☐	El código está separado en componentes, servicios y tipos.
☐	El README explica cómo ejecutar el proyecto.

14. Formato sugerido para el README

- Nombre del proyecto.
- Nombre del desarrollador.
- Descripción breve de la aplicación.
- Tecnologías utilizadas.
- Instrucciones de instalación.
- Comando para ejecutar el proyecto.
- Funcionalidades implementadas.
- Capturas de pantalla.
- Problemas encontrados y cómo los resolvieron.

15. Fuentes de referencia

- Documentación oficial de PokéAPI: <https://pokeapi.co/docs/v2>
- Sitio principal de PokéAPI: <https://pokeapi.co/>